



中南大學
CENTRAL SOUTH UNIVERSITY

信息论实验报告

学生姓名：	樊博文
学 号：	8207230322
学 院：	自动化学院
专业班级：	电子信息工程 2303

本科生院制

2026 年 5 月

目录

实验一 信源熵的计算和函数曲线绘制	2
一、实验目的	2
二、实验原理	2
三、实验内容	2
四、程序代码	3
五、实验结果及分析	6
1. 二元信源熵函数曲线结果及分析	6
2. 三元信源熵函数曲面结果及分析	6
3. 英文信源文档统计结果及分析	7
六、实验结论及体会	8
实验二 二进制香农编码	9
一、实验目的	9
二、实验原理	9
三、实验内容	10
四、程序代码	10
五、程序流程图	12
六、实验结果及分析	13
七、实验结论及体会	15
实验三 二进制和三进制霍夫曼编码	16
一、实验目的	16
二、实验原理	16
三、实验内容	17
四、程序代码	18
五、程序流程图	21
六、实验结果及分析	22
1. 二进制霍夫曼编码过程及分析	22
2. 三进制霍夫曼编码过程及分析	23
3. 二进制与三进制霍夫曼编码对比分析	23
七、实验结论及体会	24
实验四 一般离散信道容量迭代算法	25
一、实验目的	25
二、实验原理	25
三、实验内容	26
四、程序代码	26
五、实验结果及分析	29
1. 程序运行结果分析	29
2. 信道容量迭代收敛曲线分析	31
3. 算法与结果综合分析	31
六、实验结论及体会	32

实验一 信源熵的计算和函数曲线绘制

一、实验目的

1. 掌握熵函数表达式及其性质，实现信源熵的计算；
2. 掌握 MATLAB 绘图函数，分别编程绘制二元、三元熵函数曲线；
3. 实现以下功能：

输入：一篇英文字母组成的信源文档

输出：给出该信源文档中各个字母与空格的概率分布，以及该信源的熵

二、实验原理

信源熵是信息论中用来度量信源平均不确定性的物理量。对于一个离散信源，设其符号集合为：

$$X = \{x_1, x_2, \dots, x_n\}$$

各符号出现的概率为：

$$P = \{p_1, p_2, \dots, p_n\}$$

则信源熵定义为：

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i$$

其中， p_i 表示第 i 个符号出现的概率， $H(X)$ 表示信源平均每个符号所包含的信息量，单位通常为 bit/symbol。

该公式的含义是：如果某个符号出现的概率很大，那么它带来的不确定性较小；如果一个符号出现的概率较小，那么它一旦出现，所携带的信息量较大。信源熵并不是某一个符号的信息量，而是整个信源所有符号信息量的平均结果。

对于二元信源，设两个符号出现的概率分别为 p 和 $1-p$ ，则二元信源熵为：

$$H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$$

当 $p=0$ 或 $p=1$ 时，信源只会出现一种符号，没有不确定性，因此熵为 0；当 $p=0.5$ 时，两个符号等概率出现，信源的不确定性最大，熵达到最大值 1。

对于三元信源，设三个符号出现的概率分别为 p_1 、 p_2 、 p_3 ，则：

$$H(p_1, p_2, p_3) = -p_1 \log_2 p_1 - p_2 \log_2 p_2 - p_3 \log_2 p_3$$

并且满足：

$$p_1 + p_2 + p_3 = 1$$

在 MATLAB 程序中，可以令：

$$p_3 = 1 - p_1 - p_2$$

这样只需要改变 p_1 和 p_2 ，就可以计算三元信源在不同概率分布下的熵值。

在实际计算中需要注意，当某个概率为 0 时，不能直接计算 $\log_2 0$ ，否则程序会出现无穷大或错误。因此在程序中需要对概率为 0 的情况进行判断，并规定：

$$0 \log_2 0 = 0$$

三、实验内容

第一部分是绘制二元信源熵函数曲线。通过设置 p 从 0 到 1 变化，计算每一个概率值对应的熵值，并使用 MATLAB 的 plot 函数绘制曲线。通过曲线可以直观观察二元信源熵随概率变化的规律。

第二部分是绘制三元信源熵函数曲面。通过设置 p_1 和 p_2 的取值范围, 并由 $p_3 = 1 - p_1 - p_2$ 计算第三个符号的概率。只有当 $p_3 \geq 0$ 时, 该组概率才是合法的概率分布。然后计算对应的三元熵, 并使用 surf 函数绘制三维曲面。

第三部分是对英文信源文档进行统计。程序读取 source.txt 文件, 将其中的大写字母转换为小写字母, 然后统计 26 个英文字母和空格的出现次数。根据频数计算概率分布, 最后利用信源熵公式计算该文档的信源熵, 并绘制概率分布柱状图。

四、程序代码

```
clc;
clear;
close all;

% 一、绘制二元熵函数曲线
%  $H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$ 

p = 0:0.001:1;

H2 = zeros(size(p));

for i = 1:length(p)
    if p(i) == 0 || p(i) == 1
        H2(i) = 0;
    else
        H2(i) = -p(i)*log2(p(i)) - (1-p(i))*log2(1-p(i));
    end
end

figure;
plot(p, H2, 'LineWidth', 2);
grid on;
xlabel('p');
ylabel('H(p)');
title('二元信源熵函数曲线');

% 二、绘制三元熵函数曲面
%  $H(p_1, p_2, p_3) = -\sum(p_i \log_2 p_i)$ 
% 其中  $p_3 = 1 - p_1 - p_2$ 

step = 0.01;
p1 = 0:step:1;
p2 = 0:step:1;

[P1, P2] = meshgrid(p1, p2);
P3 = 1 - P1 - P2;

H3 = zeros(size(P1));

for i = 1:size(P1, 1)
    for j = 1:size(P1, 2)
        if P3(i, j) >= 0
            probs = [P1(i, j), P2(i, j), P3(i, j)];
            H_temp = 0;

            for k = 1:3
```

```
        if probs(k) > 0
            H_temp = H_temp - probs(k) * log2(probs(k));
        end
    end

    H3(i, j) = H_temp;
else
    H3(i, j) = NaN;
end
end
end

figure;
surf(P1, P2, H3);
shading interp;
grid on;
xlabel('p_1');
ylabel('p_2');
zlabel('H(p_1,p_2,p_3)');
title('三元信源熵函数曲面');

% 三、统计英文文档中字母与空格概率分布及信源熵

filename = 'source.txt';

fid = fopen(filename, 'r');

if fid == -1
    error('无法打开文件，请确认 source.txt 是否在当前工作目录下。');
end

text = fread(fid, '*char');
fclose(fid);

% 转换为小写
text = lower(text);

% 统计对象：26 个英文字母 + 空格
symbols = ['a':'z', ' '];
counts = zeros(1, length(symbols));

% 统计频数
for i = 1:length(text)
    ch = text(i);

    for j = 1:length(symbols)
        if ch == symbols(j)
            counts(j) = counts(j) + 1;
            break;
        end
    end
end

% 总符号数
total_count = sum(counts);
```

```

if total_count == 0
    error('文档中没有英文字母或空格。');
end

% 计算概率分布
probabilities = counts / total_count;

% 计算信源熵
H_source = 0;

for i = 1:length(probabilities)
    if probabilities(i) > 0
        H_source = H_source - probabilities(i) * log2(probabilities(i));
    end
end

% 输出统计结果

fprintf('\n 英文信源文档统计结果\n');
fprintf('总符号数: %d\n\n', total_count);

fprintf('符号\t 频数\t 概率\n');
fprintf('-----\n');

for i = 1:length(symbols)
    if symbols(i) == ' '
        fprintf('空格\t%d\t%.6f\n', counts(i), probabilities(i));
    else
        fprintf('%c\t %d\t %.6f\n', symbols(i), counts(i), probabilities(i));
    end
end

fprintf('\n 该信源的熵 H = %.6f bit/symbol\n', H_source);

% 绘制概率分布柱状图

figure;

bar(probabilities);
grid on;

xticks(1:length(symbols));

symbol_labels = cell(1, length(symbols));
for i = 1:length(symbols)
    if symbols(i) == ' '
        symbol_labels{i} = 'space';
    else
        symbol_labels{i} = symbols(i);
    end
end

xticklabels(symbol_labels);

xlabel('符号');
ylabel('概率');
title('英文信源中字母与空格的概率分布');

```

五、实验结果及分析

1. 二元信源熵函数曲线结果及分析

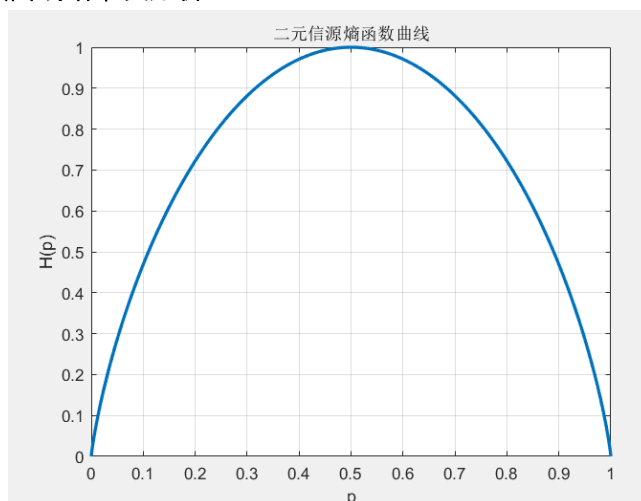


图 1-1 二元信源熵函数曲线

由图 1-1 曲线可以看出，横坐标为概率 p ，纵坐标为对应的熵值 $H(p)$ 。曲线整体呈现先增大后减小的趋势，并且关于 $p = 0.5$ 对称。

当 $p = 0$ 或 $p = 1$ 时，二元信源中实际上只有一个符号会出现，信源输出是完全确定的，因此此时熵为 0。当 $p = 0.5$ 时，两个符号出现的概率相等，接收者最难判断下一个符号是什么，信源的不确定性最大，因此熵达到最大值：

$$H_{\max} = 1 \text{ bit/symbol}$$

通过这个图像可以更直观地理解熵的含义：熵不是简单地由符号个数决定，而是由符号的概率分布决定。对于二元信源来说，两个符号越接近等概率，信源熵越大；如果某一个符号出现的概率越接近 1，信源越容易预测，熵就越小。

2. 三元信源熵函数曲面结果及分析

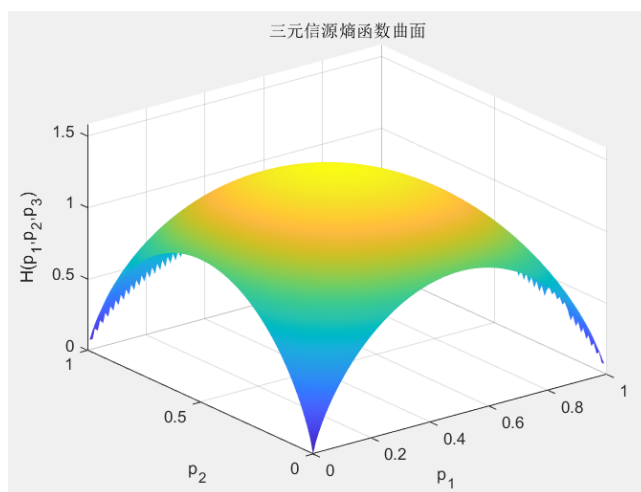


图 1-2 三元信源熵函数曲面

由图 1-2 曲面可以看出，三元信源熵随着 p_1 、 p_2 、 p_3 的变化而变化，其中：

$$p_3 = 1 - p_1 - p_2$$

并且只有满足： $p_1 \geq 0, p_2 \geq 0, p_3 \geq 0$ 的概率组合才有实际意义。

从实验图像中可以看到，曲面中间区域的熵值较大，边缘区域的熵值较小。这说明当三个符号的概率比较接近时，信源的不确定性较大；当概率分布比较集中，即某一个符号出现概率明显大于其他符号时，信源的不确定性减小。

三元信源熵的最大值出现在三个符号等概率时：

$$p_1 = p_2 = p_3 = \frac{1}{3}$$

此时： $H_{\max} = \log_2 3 \approx 1.585 \text{ bit/symbol}$

从三元熵函数曲面中可以进一步看出，信源熵的大小本质上反映的是概率分布的均匀程度。概率越均匀，信源越难预测，熵越大；概率越集中，信源越容易预测，熵越小。

3. 英文信源文档统计结果及分析

本次实验的输入信源文档为：

this is an example of information theory
knowledge is power

命令行窗口

英文信源文档统计结果		
总符号数：58		
符号	频数	概率

a	3	0.051724
b	0	0.000000
c	0	0.000000
d	1	0.017241
e	6	0.103448
f	2	0.034483
g	1	0.017241
h	2	0.034483
i	5	0.086207
j	0	0.000000
k	1	0.017241
l	2	0.034483
m	2	0.034483
n	4	0.068966
o	6	0.103448
p	2	0.034483
q	0	0.000000
r	3	0.051724
s	3	0.051724
t	3	0.051724
u	0	0.000000
v	0	0.000000
w	2	0.034483
x	1	0.017241
y	1	0.017241
z	0	0.000000
空格	8	0.137931

该信源的熵 $H = 4.036451 \text{ bit/symbol}$

图 1-3 运行结果

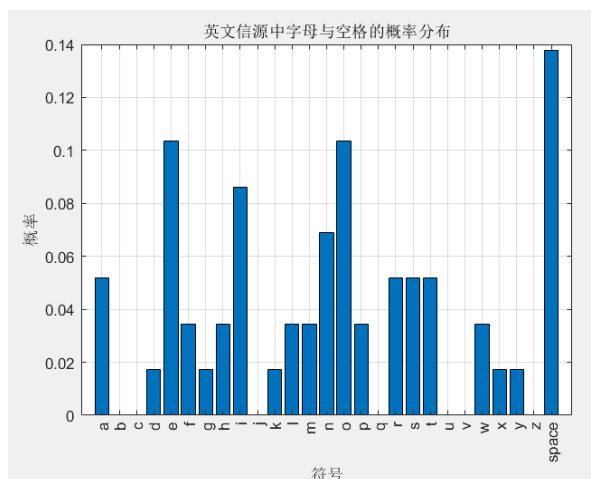


图 1-4 英文信源统计概率分布柱状图

对输入英文信源文档进行统计后，得到总符号数为：58

其中统计对象包括 26 个英文字母和空格。程序计算得到该信源的熵为：

$$H = 4.036451 \text{ bit/symbol}$$

从图 1-4 可以看出，在该英文信源文档中，空格出现次数最多，为 8 次，概率为 0.137931；字母 e 和 o 出现次数也较多，均为 6 次，概率为 0.103448；字母 i 出现 5 次，概率为 0.086207；字母 n 出现 4 次，概率为 0.068966。而 b、c、j、q、u、v、z 等字母在该文档中没有出现，概率为 0。

这说明该英文信源的符号分布并不是均匀的。虽然统计对象共有 27 个符号, 即 26 个字母和 1 个空格, 但每个符号出现的概率差异较大。如果这 27 个符号完全等概率出现, 则最大熵为:

$$H_{\max} = \log_2 27 \approx 4.755$$

而本实验计算得到的信源熵为:

$$H = 4.036451 \text{ bit/symbol}$$

该值小于最大熵, 说明该英文文本具有一定的统计规律, 并不是完全随机的等概率信源。

我对这一结果的理解是: 英文文本中不同字母的使用频率本身就不相同, 空格和常用字母出现概率较高, 而一些字母很少出现。因此, 英文信源具有一定的可预测性。信源熵越小, 说明信源越有规律; 信源熵越大, 说明信源越接近随机。实验结果也说明, 信源熵可以用一个具体的数值来描述文本信源的平均不确定性。

六、实验结论及体会

通过本次实验, 我完成了信源熵的计算、二元熵函数曲线绘制、三元熵函数曲面绘制以及英文信源文档的概率统计与熵计算。实验结果验证了信源熵的基本性质: 信源概率分布越均匀, 熵越大; 概率分布越集中, 熵越小。

对于二元信源, 当 $p = 0.5$ 时, 两个符号出现的概率相等, 信源不确定性最大, 熵达到最大值 1 bit/symbol; 当 $p = 0$ 或 $p = 1$ 时, 信源输出完全确定, 熵为 0。通过二元熵函数曲线, 我更加直观地理解了熵与概率之间的关系。

对于三元信源, 当三个符号等概率出现时, 熵达到最大值 $\log_2 3$ 。通过三元熵函数曲面可以看出, 熵的最大值出现在概率分布最均匀的位置, 而不是某个符号概率特别大的位置。这说明熵反映的是整个信源的不确定性, 而不是单个符号的信息量大小。

在英文信源文档统计部分, 程序对文本进行了字母和空格统计, 得到总符号数为 58, 信源熵为:

$$H = 4.036451 \text{ bit/symbol}$$

该值小于 27 个符号等概率分布时的最大熵, 说明实际英文文本的符号分布并不均匀, 具有一定规律性。这也让我理解到, 信息论中的熵不仅是一个理论公式, 也可以用来分析实际文本信源的统计特征。

在编写程序计算熵时, 需要特别注意概率为 0 的情况。因为 $\log_2 0$ 在数学和程序计算中没有意义, 所以程序中必须先判断概率是否大于 0, 再进行熵的累加计算。这说明在把理论公式转化为程序时, 不能只照搬公式, 还要考虑实际计算中的边界情况。

总体来说, 本次实验加深了我对信源熵概念的理解。通过图像观察和实际文本计算, 我认识到熵可以用来衡量信源平均不确定性, 也可以反映信源符号分布的均匀程度。同时, 本实验也提高了我使用 MATLAB 进行函数绘图、文件读取、字符统计和数据分析的能力。

实验二 二进制香农编码

一、实验目的

1. 总结香农编码的基本步骤，画出程序流程图；
2. 给定以下信源，实现香农编码的 Matlab 源程序。

对给定信源

$$\begin{bmatrix} X \\ P(X) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ 0.2 & 0.19 & 0.18 & 0.17 & 0.15 & 0.1 & 0.01 \end{bmatrix}$$

进行二进制香农编码，通过 MATLAB 进行编码过程仿真，并计算平均码长及编码效率。

二、实验原理

香农编码是一种根据信源符号概率构造变长码的方法。其基本思想是：出现概率大的符号分配较短的码字，出现概率小的符号分配较长的码字，从而降低平均码长，提高编码效率。

设离散信源为：

$$X = \{a_1, a_2, \dots, a_n\}$$

对应概率分布为：

$$P = \{p_1, p_2, \dots, p_n\}$$

其中：

$$\sum_{i=1}^n p_i = 1$$

香农编码通常先将信源符号按概率从大到小排序，然后计算每个符号的码长：

$$l_i = \lceil -\log_2 p_i \rceil$$

其中， $\lceil \cdot \rceil$ 表示向上取整。

接着计算每个符号的累加概率：

$$F_i = \sum_{k=1}^{i-1} p_k$$

对于第一个符号，有：

$$F_1 = 0$$

然后将累加概率 F_i 转换成二进制小数，取其小数点后的前 l_i 位作为该符号的香农码字。

信源熵的计算公式为：

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i$$

平均码长为：

$$\bar{L} = \sum_{i=1}^n p_i l_i$$

编码效率为：

$$\eta = \frac{H(X)}{\bar{L}} \times 100\%$$

香农编码不是直接让所有符号使用相同长度的码字,而是根据信源概率的不同来分配不同长度的码字。概率越大的符号在实际传输中出现得越频繁,因此使用较短码字可以减少整体平均码长;概率越小的符号虽然码字较长,但出现次数少,对平均码长影响较小。这体现了信源编码中常见符号短编码,少见符号长编码的思想。

三、实验内容

本实验给定信源:

$$\begin{bmatrix} X \\ P(X) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ 0.2 & 0.19 & 0.18 & 0.17 & 0.15 & 0.1 & 0.01 \end{bmatrix}$$

要求对该信源进行二进制香农编码,并完成以下内容:

1. 输入信源符号及其概率。
2. 将信源符号按照概率从大到小排序。
3. 根据公式 $l_i = \lceil -\log_2 p_i \rceil$ 计算每个符号的码长。
4. 计算每个符号的累加概率 F_i 。
5. 将累加概率转换为二进制小数,并取前 l_i 位作为码字。
6. 输出每个符号对应的概率、累加概率、码长和码字。
7. 计算信源熵、平均码长和编码效率。

四、程序代码

```
clc;
clear;
close all;

% 实验二: 二进制香农编码

% 信源符号
symbols = {'a1','a2','a3','a4','a5','a6','a7'};

% 信源概率
P = [0.2 0.19 0.18 0.17 0.15 0.1 0.01];

% 按概率从大到小排序
[P_sort, index] = sort(P, 'descend');
symbols_sort = symbols(index);

n = length(P_sort);

% 计算码长 li = ceil(-log2(pi))
L = ceil(-log2(P_sort));

% 计算累加概率 Fi
F = zeros(1, n);
for i = 2:n
    F(i) = F(i-1) + P_sort(i-1);
end

% 生成香农编码
codes = cell(1, n);
```

```

for i = 1:n
    temp = F(i);
    code = "";

    for j = 1:L(i)
        temp = temp * 2;

        if temp >= 1
            code = [code '1'];
            temp = temp - 1;
        else
            code = [code '0'];
        end
    end

    codes{i} = code;
end

% 计算信源熵
H = 0;
for i = 1:n
    H = H - P_sort(i) * log2(P_sort(i));
end

% 计算平均码长
L_avg = sum(P_sort .* L);

% 计算编码效率
eta = H / L_avg;

% 输出结果

fprintf('二进制香农编码结果\n\n');

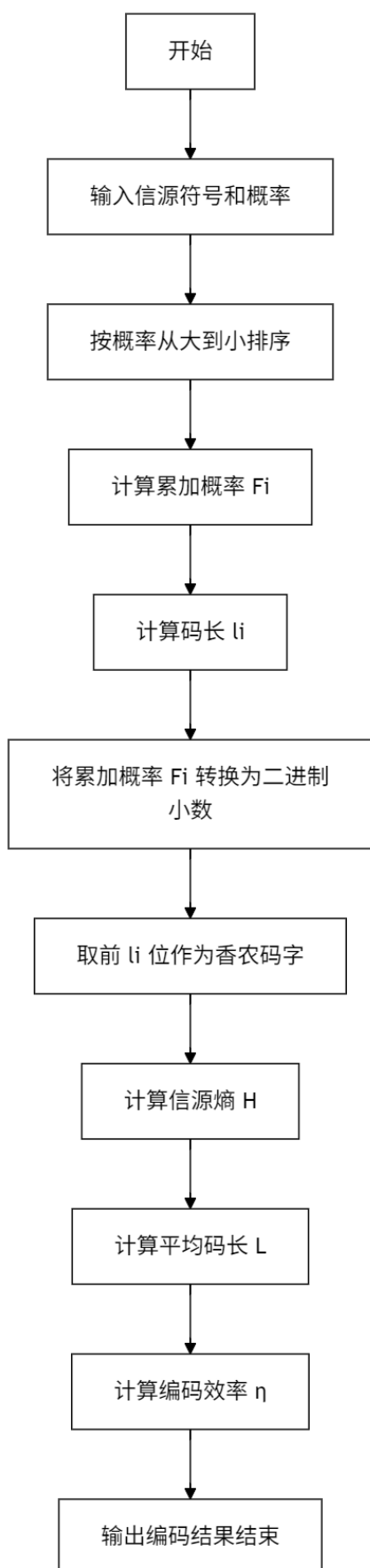
fprintf('%-7s %-8s %-9s %-7s %-10s\n', ...
    '符号', '概率', '累加概率', '码长', '码字');
fprintf('-----\n');

for i = 1:n
    fprintf('%-8s %-10.2f %-12.2f %-8d %-10s\n', ...
        symbols_sort{i}, P_sort(i), F(i), L(i), codes{i});
end

fprintf('\n 信源熵 H(X)    = %.6f bit/symbol\n', H);
fprintf('平均码长 L      = %.6f bit/symbol\n', L_avg);
fprintf('编码效率 η      = %.2f%%\n', eta * 100);

```

五、程序流程图



如图 2-1 二进制香农编码的程序流程主要包括输入信源、概率排序、计算累加概率、计算码长、生成码字以及计算编码性能指标几个部分。

首先，程序输入给定信源符号及其对应概率。本实验中的信源共有 7 个符号，分别为 $a_1, a_2, \dots, a_7$ ，对应概率为：

0.2, 0.19, 0.18, 0.17, 0.15, 0.1, 0.01

然后，程序按照概率从大到小对信源符号进行排序。排序的目的是保证概率较大的符号优先编码，这符合香农编码中“高概率符号分配短码字，低概率符号分配长码字”的思想。本实验给定的概率本身已经是从小到大排列的，因此排序后符号顺序没有发生改变。

接着，程序计算每个符号的累加概率 F_i 。累加概率的计算公式为：

$$F_i = \sum_{k=1}^{i-1} p_k$$

其中第一个符号的累加概率为：

$$F_1 = 0$$

累加概率是生成香农码字的重要依据，后续需要将其转换为二进制小数。

之后，程序根据公式：

$$l_i = \lceil -\log_2 p_i \rceil$$

计算每个信源符号的码长。由于码长必须是整数，所以需要对 $-\log_2 p_i$ 向上取整。这样可以保证所得码字长度满足香农编码的要求。

随后，程序将每个符号的累加概率 F_i 转换为二进制小数，并取小数点后的前 l_i 位作为该符号的码字。具体实现时，程序通过不断将累加概率乘以 2 来判断当前二进制位是 0 还是 1：如果乘 2 后大于等于 1，则该位为 1，并减去 1；否则该位为 0。循环执行 l_i 次后，就得到对应长度的香农码字。

最后，程序根据概率分布计算信源熵、平均码长和编码效率，并输出各符号的概率、累加概率、码长、码字以及整体编码性能指标。

整个流程体现了香农编码的基本思路：先根据信源概率确定码长，再利用累加概率生成满足前缀条件的二进制码字，最后通过平均码长和编码效率评价编码效果。

图 2-1 二进制香农编码总体流程图

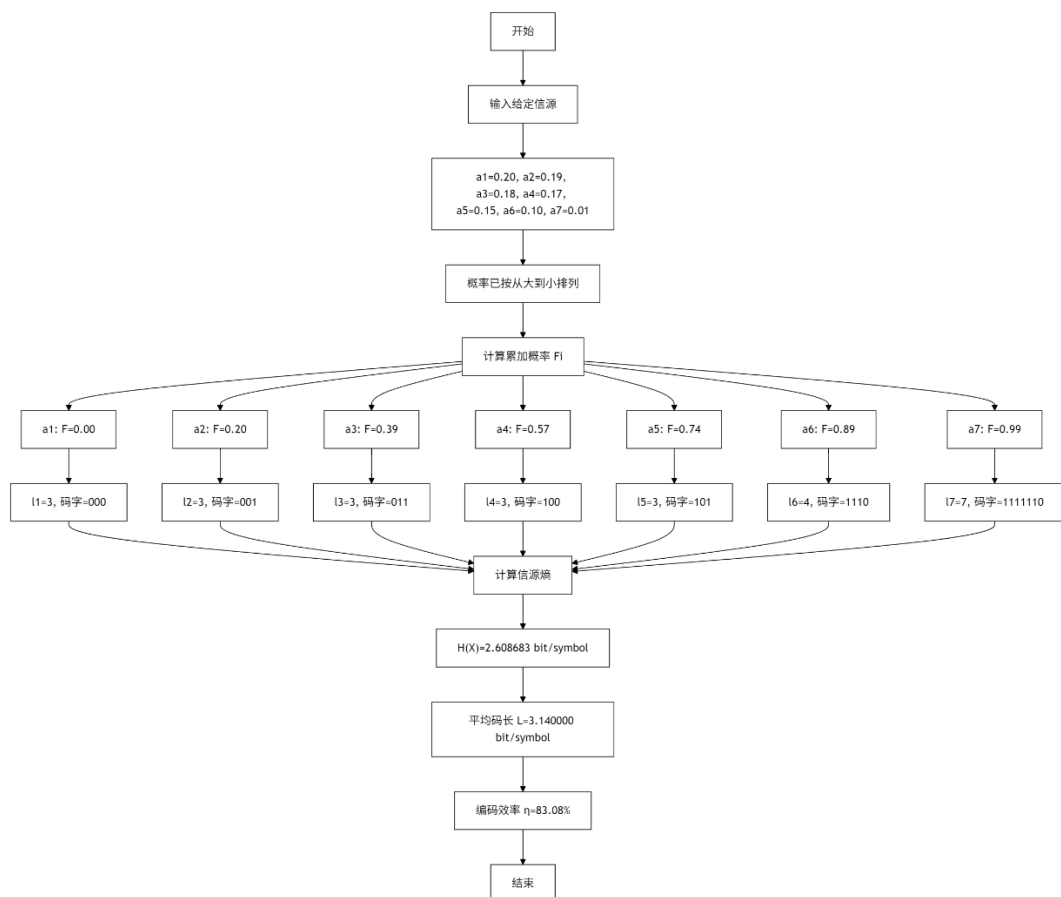


图 2-2 本实验香农编码过程

图 2-2 是在二进制香农编码总体流程的基础上，结合本实验给定信源得到的具体编码过程，可以清楚地看到每个符号从概率输入、累加概率计算、码长确定到码字生成的完整过程。

六、实验结果及分析

程序运行后，得到如下二进制香农编码结果：

命令行窗口				
二进制香农编码结果				
符号	概率	累加概率	码长	码字
a1	0.20	0.00	3	000
a2	0.19	0.20	3	001
a3	0.18	0.39	3	011
a4	0.17	0.57	3	100
a5	0.15	0.74	3	101
a6	0.10	0.89	4	1110
a7	0.01	0.99	7	1111110
信源熵 $H(X)$	= 2.608683 bit/symbol			
平均码长 L	= 3.140000 bit/symbol			
编码效率 η	= 83.08%			

图 2-3 二进制香农编码运行结果

由图 2-3 可知，各信源符号的编码结果如下表所示：

符号	概率	累加概率	码长	码字
a_1	0.20	0.00	3	000
a_2	0.19	0.20	3	001
a_3	0.18	0.39	3	011
a_4	0.17	0.57	3	100
a_5	0.15	0.74	3	101
a_6	0.10	0.89	4	1110
a_7	0.01	0.99	7	1111110

程序计算得到：

$$H(X) = 2.608683 \text{ bit/symbol}$$

$$\bar{L} = 3.140000 \text{ bit/symbol}$$

$$\eta = 83.08\%$$

从编码结果可以看出，概率较大的符号 $a_1 \sim a_5$ 的码长均为 3，而概率较小的 a_6 码长为 4，概率最小的 a_7 码长为 7。这说明香农编码能够根据信源符号出现概率的大小分配不同长度的码字。概率越大的符号出现越频繁，因此使用较短码字；概率越小的符号出现次数较少，因此可以分配较长码字。

例如， a_1 的概率为 0.20，累加概率为 0.00，码长为 3，因此将 0.00 转换为二进制小数并取前三位，得到码字 000。 a_7 的概率为 0.01，累加概率为 0.99，码长为 7，因此得到码字 1111110。由于 a_7 出现概率很小，所以虽然它的码字较长，但对平均码长的影响相对有限。

信源熵为：

$$H(X) = 2.608683 \text{ bit/symbol}$$

它表示该信源平均每个符号所包含的信息量，也是理论上进行无失真编码时平均码长能够接近的下限。实验中得到的平均码长为：

$$\bar{L} = 3.140000 \text{ bit/symbol}$$

可以看出：

$$\bar{L} > H(X)$$

这符合信源编码的基本规律。因为实际编码时，码长必须取整数，而理想信息量 $-\log_2 p_i$ 通常不是整数，所以香农编码会产生一定的冗余。

编码效率为：

$$\eta = \frac{H(X)}{\bar{L}} \times 100\% = 83.08\%$$

该结果说明本次二进制香农编码能够较好地利用信源概率分布，但仍然没有达到理论最优。造成编码效率小于 100% 的主要原因是码长向上取整，使得实际平均码长大于信源熵。

从本实验结果也可以看出，香农编码不只是简单地给每个符号分配一个编号，而是根据信源的概率特性进行编码。它的核心思想是尽量让常出现的符号使用较短码字，从而降低整体平均码长。虽然本实验中的平均码长为 3.14 bit/symbol，仍大于信源熵，但已经体现了变长编码的基本思想。

七、实验结论及体会

按照要求对给定信源的二进制香农编码，并利用 MATLAB 程序实现了编码过程仿真。程序输出了每个信源符号的概率、累加概率、码长和码字，同时计算出了信源熵、平均码长和编码效率。

本次实验得到的主要结果为：

$$H(X) = 2.608683 \text{ bit/symbol}$$

$$\bar{L} = 3.140000 \text{ bit/symbol}$$

$$\eta = 83.08\%$$

实验结果表明，香农编码能够根据信源符号概率分配不同长度的码字。概率较大的符号分配较短码字，概率较小的符号分配较长码字，这符合信源编码中提高编码效率的基本思想。

通过本次实验，我进一步理解了信源熵和平均码长之间的关系。信源熵表示理论上的平均信息量下限，而平均码长表示实际编码后每个符号平均所需的二进制位数。一个编码方法越有效，其平均码长就越接近信源熵。本实验中平均码长大于信源熵，说明香农编码虽然能够接近理论极限，但由于码长取整等原因，仍存在一定冗余。

在程序实现过程中，我认为最关键的步骤是累加概率的计算和二进制码字的生成。累加概率决定了每个符号码字的起始位置，而码长决定了最终需要截取多少位二进制小数。通过 MATLAB 循环模拟二进制小数转换过程，可以清楚地看到香农码字是如何一步步生成的。

本实验也让我认识到，编码并不是随意给符号编号，而是要结合信源概率分布进行设计。只有充分利用信源的统计特性，才能有效减少平均码长，提高编码效率。香农编码虽然不是最优编码方法，但它具有明确的理论依据和较直观的实现过程，是理解信源编码思想的重要基础。总体来说，本次实验加深了我对二进制香农编码原理、实现步骤和性能评价方法的理解，也提高了我使用 MATLAB 进行信息论编码仿真的能力。

实验三 二进制和三进制霍夫曼编码

一、实验目的

1. 总结二进制霍夫曼编码的基本步骤，画出程序流程图；
2. 给定以下信源，实现霍夫曼编码的 Matlab 源程序。

对给定信源

$$\begin{bmatrix} X \\ P(X) \end{bmatrix} = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ 0.2 & 0.19 & 0.18 & 0.17 & 0.15 & 0.1 & 0.01 \end{bmatrix}$$

进行二进制和三进制霍夫曼编码，通过 MATLAB 进行编码过程仿真，并计算平均码长及编码效率。

二、实验原理

霍夫曼编码是一种常用的变长编码方法。它的基本思想是根据信源符号出现概率的不同，为不同符号分配不同长度的码字。一般来说，出现概率大的符号分配较短的码字，出现概率小的符号分配较长的码字，从而减小平均码长，提高编码效率。

1. 二进制霍夫曼编码原理

二进制霍夫曼编码每次从当前节点集合中选择概率最小的两个节点进行合并，合并后生成一个新节点，新节点的概率等于两个节点概率之和。然后将新节点重新放回集合中，再次排序、合并，直到所有节点合并成一棵完整的霍夫曼树。

二进制霍夫曼编码的基本步骤如下：

1. 将信源符号按照概率从小到大排列；
2. 选取概率最小的两个节点进行合并；
3. 合并后的新节点概率为两个节点概率之和；
4. 将新节点放回节点集合中；
5. 重复排序和合并过程，直到只剩一个根节点；
6. 从根节点开始，对二叉树的两个分支分别赋值为 0 和 1；
7. 从根节点到叶节点的路径即为该信源符号的霍夫曼码字。

信源熵计算公式为：

$$H(X) = - \sum_{i=1}^n p_i \log_2 p_i$$

平均码长为：

$$\bar{L} = \sum_{i=1}^n p_i l_i$$

编码效率为：

$$\eta = \frac{H(X)}{\bar{L}} \times 100\%$$

其中， p_i 表示第 i 个信源符号的概率， l_i 表示第 i 个信源符号对应码字的长度。

2. 三进制霍夫曼编码原理

三进制霍夫曼编码与二进制霍夫曼编码的思想基本相同, 不同之处在于三进制霍夫曼编码每次选择概率最小的三个节点进行合并, 并且每个节点有三个分支, 分别赋值为 0、1、2。

对于 D 进制霍夫曼编码, 信源符号个数需要满足:

$$n \equiv 1(\text{mod} D - 1)$$

如果不满足该条件, 就需要补充若干个概率为 0 的虚符号, 使其满足构造 D 进制霍夫曼树的条件。

本实验中进行三进制霍夫曼编码时, $D = 3$, 信源符号数为:

$$n = 7$$

因为:

$$7 \equiv 1(\text{mod} 2)$$

所以本实验进行三进制霍夫曼编码时不需要补充虚符号。

三进制信源熵为:

$$H_3(X) = - \sum_{i=1}^n p_i \log_3 p_i$$

也可以由二进制信源熵换算得到:

$$H_3(X) = \frac{H(X)}{\log_2 3}$$

三进制平均码长为:

$$\bar{L}_3 = \sum_{i=1}^n p_i l_i$$

三进制编码效率为:

$$\eta_3 = \frac{H_3(X)}{\bar{L}_3} \times 100\%$$

霍夫曼编码通过不断合并小概率节点, 把小概率符号放在树的较深位置, 使其码字较长; 把大概率符号放在树的较浅位置, 使其码字较短。这样虽然每个符号的码长不同, 但从整体平均来看, 可以减少信源的平均编码长度。

三、实验内容

本实验对给定信源分别进行二进制和三进制霍夫曼编码, 主要完成以下内容:

1. 输入信源符号及对应概率;
2. 编写通用的 D 进制霍夫曼编码函数;
3. 当 $D = 2$ 时, 实现二进制霍夫曼编码;
4. 当 $D = 3$ 时, 实现三进制霍夫曼编码;
5. 输出二进制和三进制霍夫曼编码的合并过程;
6. 输出各信源符号对应的码字和码长;
7. 计算二进制霍夫曼编码的信源熵、平均码长和编码效率;
8. 计算三进制霍夫曼编码的信源熵、平均码长和编码效率;
9. 对二进制和三进制霍夫曼编码结果进行比较和分析。

四、程序代码

```

clc;
clear;
close all;

% 实验三：二进制和三进制霍夫曼编码

% 信源符号
symbols = {'a1','a2','a3','a4','a5','a6','a7'};

% 信源概率
P = [0.2 0.19 0.18 0.17 0.15 0.1 0.01];

%% 计算二进制霍夫曼编码

[ codes2, len2, process2 ] = huffman_code(symbols, P, 2);

% 信源熵, 单位 bit/symbol
H2 = -sum(P .* log2(P));

% 二进制平均码长, 单位 bit/symbol
L_avg2 = sum(P .* len2);

% 二进制编码效率
eta2 = H2 / L_avg2;

%% 计算三进制霍夫曼编码

[ codes3, len3, process3 ] = huffman_code(symbols, P, 3);

% 三进制信源熵, 单位 trit/symbol
H3 = H2 / log2(3);

% 三进制平均码长, 单位 trit/symbol
L_avg3 = sum(P .* len3);

% 三进制编码效率
eta3 = H3 / L_avg3;

%% 输出二进制霍夫曼编码过程

fprintf('二进制霍夫曼编码合并过程\n\n');

for i = 1:length(process2)
    fprintf('第%-2d 次合并:  %-35s  合并后概率 = %.2f\n', ...
        i, process2(i).items, process2(i).weight);
end

fprintf('\n 二进制霍夫曼编码结果\n\n');
fprintf('%-7s  %-8s  %-10s  %-8s\n', '符号', '概率', '码字', '码长');
fprintf('-----\n');

for i = 1:length(symbols)
    fprintf('%-8s  %-10.2f  %-12s  %-8d\n', ...
        symbols{i}, P(i), codes2{i}, len2(i));
end

```

```

fprintf('\n 信源熵 H(X)          = %.6f bit/symbol\n', H2);
fprintf('二进制平均码长 L      = %.6f bit/symbol\n', L_avg2);
fprintf('二进制编码效率  $\eta$     = %.2f%%\n', eta2 * 100);

%% 输出三进制霍夫曼编码过程

fprintf('\n\n 三进制霍夫曼编码合并过程\n\n');

for i = 1:length(process3)
    fprintf('第%-2d 次合并:  %-35s  合并后概率 = %.2f\n', ...
        i, process3(i).items, process3(i).weight);
end

fprintf('\n 三进制霍夫曼编码结果\n\n');
fprintf('%-7s %-8s %-10s %-8s\n', '符号', '概率', '码字', '码长');
fprintf('-----\n');

for i = 1:length(symbols)
    fprintf('%-8s %-10.2f %-12s %-8d\n', ...
        symbols{i}, P(i), codes3{i}, len3(i));
end

fprintf('\n 信源熵 H(X)          = %.6f bit/symbol\n', H2);
fprintf('三进制信源熵 H3(X) = %.6f trit/symbol\n', H3);
fprintf('三进制平均码长 L      = %.6f trit/symbol\n', L_avg3);
fprintf('三进制编码效率  $\eta$     = %.2f%%\n', eta3 * 100);

% D 进制霍夫曼编码函数

function [codes, lengths, process] = huffman_code(symbols, P, D)

    n = length(P);

    % 初始化码字
    codes = cell(1, n);
    for i = 1:n
        codes{i} = '';
    end

    % 判断是否需要补充虚符号
    padNum = mod((D - 1) - mod(n - 1, D - 1), D - 1);

    % 初始化节点
    nodes = struct('prob', {}, 'index', {}, 'name', {});

    for i = 1:n
        nodes(i).prob = P(i);
        nodes(i).index = i;
        nodes(i).name = symbols{i};
    end

    % 补充概率为 0 的虚符号
    for i = 1:padNum
        nodes(n + i).prob = 0;
        nodes(n + i).index = [];
        nodes(n + i).name = ['dummy' num2str(i)];
    end

```

```

end

process = struct('items', {}, 'weight', {});

step = 0;

% 霍夫曼合并过程
while length(nodes) > 1

    % 按概率从小到大排序
    [~, order] = sort([nodes.prob], 'ascend');
    nodes = nodes(order);

    % 取概率最小的 D 个节点
    selected = nodes(1:D);

    % 给 D 个分支分别赋 0,1,2,...
    for k = 1:D
        digit = num2str(k - 1);
        idx = selected(k).index;

        for m = 1:length(idx)
            codes{idx(m)} = [digit codes{idx(m)}];
        end
    end

    % 记录合并过程
    step = step + 1;

    nameList = cell(1, D);
    for k = 1:D
        nameList{k} = selected(k).name;
    end

    process(step).items = strjoin(nameList, ' ');
    process(step).weight = sum([selected.prob]);

    % 生成新节点
    newNode.prob = sum([selected.prob]);
    newNode.index = [selected.index];
    newNode.name = ['(' strjoin(nameList, ' ') ')'];

    % 删除已合并节点，并加入新节点
    nodes = [nodes(D+1:end), newNode];
end

% 计算码长
lengths = zeros(1, n);
for i = 1:n
    lengths(i) = length(codes{i});
end
end

```

五、程序流程图

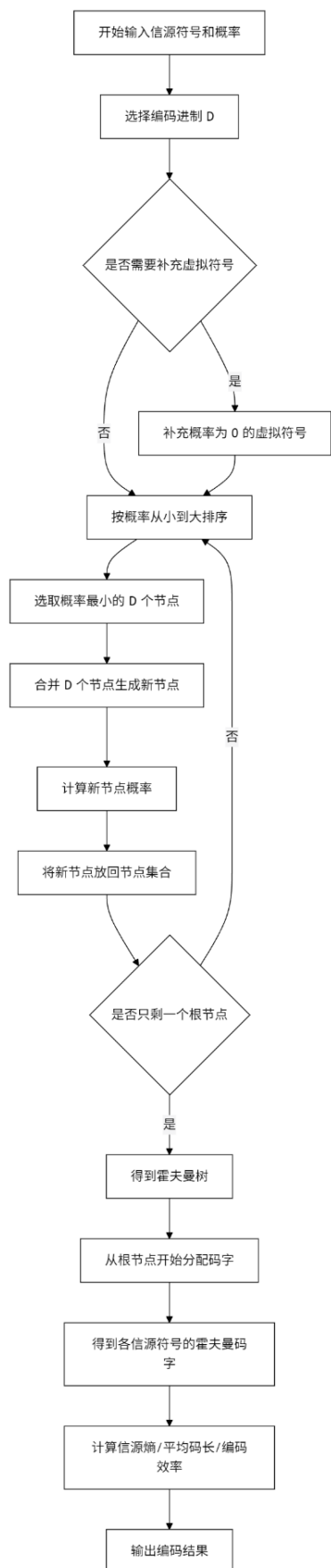


图 3-1 为二进制和三进制霍夫曼编码的总体程序流程图，展示了从输入信源到输出编码结果的完整过程。该流程图不仅适用于二进制霍夫曼编码，也适用于三进制霍夫曼编码，区别主要体现在编码进制 D 的取值不同。当 $D = 2$ 时，每次合并两个概率最小的节点；当 $D = 3$ 时，每次合并三个概率最小的节点。

程序首先输入信源符号和对应概率。输入完成后，程序选择编码进制 D 。如果进行二进制霍夫曼编码，则令 $D = 2$ 。不同进制下，每次合并的节点个数不同，最终得到的霍夫曼树结构和码字也不同。

接着，程序判断是否需要补充虚符号。对于 D 进制霍夫曼编码，信源符号个数 n 需要满足：

$$n \equiv 1(\text{mod } D - 1)$$

如果不满足该条件，就需要补充若干个概率为 0 的虚符号，使得节点个数满足构造 D 进制霍夫曼树的要求。虚符号只参与构造霍夫曼树，不作为实际信源符号输出，因此不会影响信源的平均码长和编码效率。本实验中，信源符号数为 7。对于二进制编码， $D = 2$ ，该条件恒成立；对于三进制编码， $D = 3$ ，有： $7 \equiv 1(\text{mod } 2)$ 因此本实验不需要补充虚符号。

随后，程序将所有节点按照概率从小到大排序，并选取概率最小的 D 个节点进行合并。合并后生成一个新的节点，新节点的概率等于被合并节点概率之和。例如在二进制霍夫曼编码中，第一次选取概率最小的 a_7 和 a_6 合并，得到新节点概率： $0.01 + 0.10 = 0.11$

生成新节点后，程序将其重新放回节点集合中，再次按照概率从小到大排序，继续选择概率最小的 D 个节点进行合并。这个过程不断重复，直到节点集合中只剩下一个根节点。此时，所有信源符号已经通过合并过程构成了一棵完整的霍夫曼树。

得到霍夫曼树后，程序从根节点开始向下分配码字。对于二进制霍夫曼编码，每个节点有两个分支，通常分别赋值为 0 和 1；对于三进制霍夫曼编码，每个节点有三个分支，分别赋值为 0、1、2。从根节点到某个叶节点所经过的分支标号依次连接起来，就是该叶节点对应信源符号的霍夫曼码字。

最后，程序根据各信源符号的码字长度进行计算，通过图 3-1 可以看出，霍夫曼编码的核心过程是排序—选取最小节点—合并—重新排序的循环。小概率符号会较早参与合并，因此在霍夫曼树中通常位于较深层，对应码字较长；大概率符号较晚参与合并，通常距离根节点较近，对应码字较短。

图 3-1 程序流程图

六、实验结果及分析

1. 二进制霍夫曼编码过程及分析

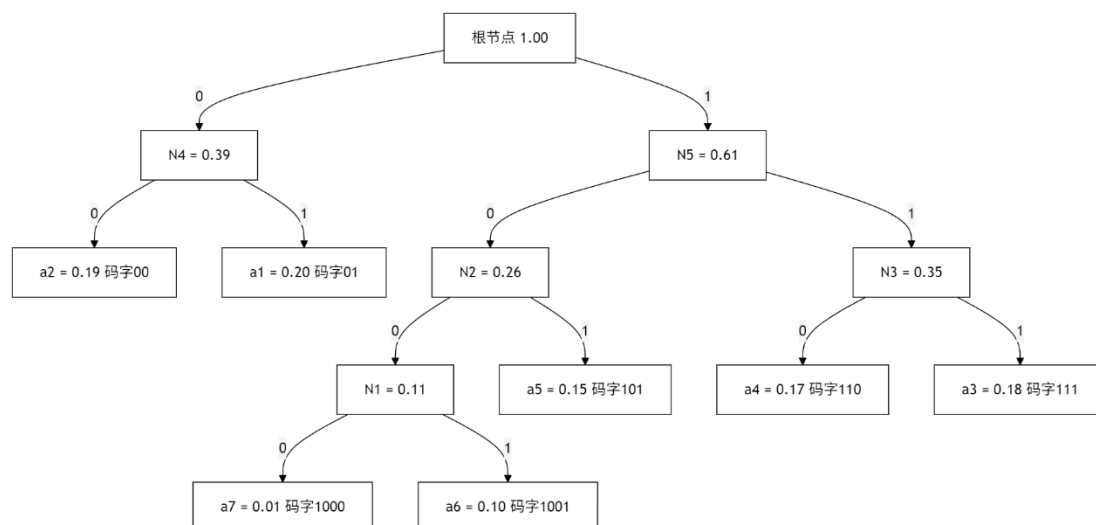


图 3-2 二进制霍夫曼编码过程

图 3-2 展示了给定信源进行二进制霍夫曼编码时的节点合并过程和霍夫曼树结构。二进制霍夫曼编码每次选取当前概率最小的两个节点进行合并，合并后的新节点概率等于这两个节点概率之和，然后再将新节点放回节点集合中继续排序和合并，直到最终形成一棵完整的二叉霍夫曼树。

命令行窗口

二进制霍夫曼编码合并过程

第1次合并: a7 + a6	合并后概率 = 0.11
第2次合并: (a7+a6) + a5	合并后概率 = 0.26
第3次合并: a4 + a3	合并后概率 = 0.35
第4次合并: a2 + a1	合并后概率 = 0.39
第5次合并: ((a7+a6)+a5) + (a4+a3)	合并后概率 = 0.61
第6次合并: (a2+a1) + (((a7+a6)+a5)+(a4+a3))	合并后概率 = 1.00

二进制霍夫曼编码结果

符号	概率	码字	码长
a1	0.20	01	2
a2	0.19	00	2
a3	0.18	111	3
a4	0.17	110	3
a5	0.15	101	3
a6	0.10	1001	4
a7	0.01	1000	4

信源熵 $H(X)$ = 2.608683 bit/symbol
 二进制平均码长 L = 2.720000 bit/symbol
 二进制编码效率 η = 95.91%

图 3-3 二进制霍夫曼编码运行结果

图 3-3 为 MATLAB 程序输出的二进制霍夫曼编码运行结果。程序输出了二进制霍夫曼编码的合并过程、各信源符号的码字和码长，并计算出信源熵、平均码长和编码效率。

2. 三进制霍夫曼编码过程及分析

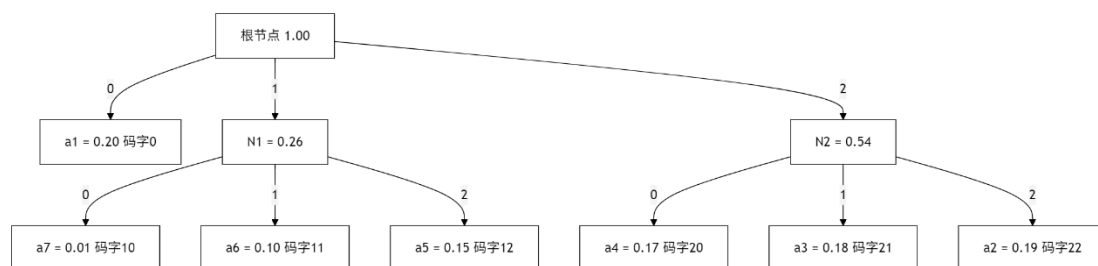


图 3-4 三进制霍夫曼编码过程

图 3-4 展示了三进制霍夫曼编码的构造过程。与二进制霍夫曼编码不同，三进制霍夫曼编码每次选择概率最小的三个节点进行合并，并且每个节点有三个分支，分别赋值为 0、1、2。

命令行窗口

三进制霍夫曼编码合并过程

第1 次合并: $a_7 + a_6 + a_5$	合并后概率 = 0.26
第2 次合并: $a_4 + a_3 + a_2$	合并后概率 = 0.54
第3 次合并: $a_1 + (a_7+a_6+a_5) + (a_4+a_3+a_2)$	合并后概率 = 1.00

三进制霍夫曼编码结果

符号	概率	码字	码长
a1	0.20	0	1
a2	0.19	22	2
a3	0.18	21	2
a4	0.17	20	2
a5	0.15	12	2
a6	0.10	11	2
a7	0.01	10	2

信源熵 $H(X)$ = 2.608683 bit/symbol
 三进制信源熵 $H_3(X)$ = 1.645896 trit/symbol
 三进制平均码长 L = 1.800000 trit/symbol
 三进制编码效率 η = 91.44%

图 3-5 三进制霍夫曼编码运行结果

图 3-5 为 MATLAB 程序输出的三进制霍夫曼编码运行结果。程序输出了三进制霍夫曼编码的合并过程、各符号对应码字、码长、三进制信源熵、三进制平均码长和编码效率。

3. 二进制与三进制霍夫曼编码对比分析

对比二进制和三进制霍夫曼编码结果可知：

编码方式	信源熵	平均码长	编码效率
二进制霍夫曼编码	2.608683 bit/symbol	2.720000 bit/symbol	95.91%
三进制霍夫曼编码	1.645896 trit/symbol	1.800000 trit/symbol	91.44%

从编码效率来看, 二进制霍夫曼编码效率为 95.91%, 三进制霍夫曼编码效率为 91.44%。在本实验给定信源下, 二进制霍夫曼编码的效率更高, 平均码长更接近对应进制下的信源熵。

但是需要注意, 二进制码长单位是 bit, 三进制码长单位是 trit, 二者不能只根据数值大小直接比较。应当在各自进制下分别用信源熵与平均码长的比值来评价编码效率。

霍夫曼编码的核心目标是让平均码长尽量接近信源熵。不同进制下的霍夫曼编码构造方式不同, 合并节点的数量不同, 最终得到的码字结构和编码效率也会不同。本实验中, 二进制霍夫曼编码的效率较高, 说明对于该组概率分布, 二叉树结构更加接近该信源的最优编码结构。

七、实验结论及体会

通过本次实验, 我完成了对给定信源的二进制和三进制霍夫曼编码, 并利用 MATLAB 程序输出了编码合并过程、各符号对应码字、平均码长和编码效率。

本实验二进制霍夫曼编码结果为:

$$H(X) = 2.608683 \text{ bit/symbol}$$

$$\bar{L} = 2.720000 \text{ bit/symbol}$$

$$\eta = 95.91\%$$

三进制霍夫曼编码结果为:

$$H_3(X) = 1.645896 \text{ trit/symbol}$$

$$\bar{L}_3 = 1.800000 \text{ trit/symbol}$$

$$\eta_3 = 91.44\%$$

实验结果说明, 霍夫曼编码能够根据信源符号的概率分布构造变长码, 使高概率符号具有较短码字, 低概率符号具有较长码字, 从而降低平均码长, 提高编码效率。

通过二进制霍夫曼编码过程可以看出, 概率较小的 a_7 和 a_6 最先合并, 因此它们在霍夫曼树中位于较深层, 码字较长; 概率较大的 a_1 和 a_2 离根节点较近, 码字较短。这让我更直观地理解了霍夫曼树结构与码字长度之间的关系。

通过三进制霍夫曼编码过程可以看出, 三进制霍夫曼编码每次合并三个概率最小的节点, 并且分支标号为 0、1、2。与二进制霍夫曼编码相比, 三进制霍夫曼树的层数较少, 但每个码元的信息单位不同, 因此在分析时需要区分 bit 和 trit, 不能简单地直接比较码长数值。

本次实验还让我认识到, 霍夫曼编码不是随意分配码字, 而是通过严格的概率排序和节点合并过程得到的。其本质是利用信源概率分布的非均匀性, 使平均码长尽可能接近信源熵。相比香农编码, 霍夫曼编码通常可以得到更短的平均码长和更高的编码效率。

总体来说, 本次实验加深了我对二进制和三进制霍夫曼编码原理的理解, 也提高了我使用 MATLAB 编写通用 D 进制霍夫曼编码程序的能力。通过观察程序流程图、编码树和程序运行结果, 我对霍夫曼编码的构造过程、码字生成方式以及编码效率评价方法有了更加清晰的认识。

实验四 一般离散信道容量迭代算法

一、实验目的

1. 掌握信道容量的概念和物理意义；
2. 掌握一般离散信道容量的迭代算法；
3. 实现以下功能：

输入：任意一个信道转移概率矩阵。信源符号个数、信宿符号个数和每个具体的转移概率在运行时从键盘输入

输出：最佳信源分布和信道容量

二、实验原理

信道容量是信息论中描述信道最大传输能力的重要物理量。对于一个离散无记忆信道，设输入符号集合为：

$$X = \{x_1, x_2, \dots, x_m\}$$

输出符号集合为：

$$Y = \{y_1, y_2, \dots, y_n\}$$

信道转移概率矩阵为：

$$P(y|x) = \begin{bmatrix} P(y_1|x_1) & P(y_2|x_1) & \cdots & P(y_n|x_1) \\ P(y_1|x_2) & P(y_2|x_2) & \cdots & P(y_n|x_2) \\ \vdots & \vdots & \ddots & \vdots \\ P(y_1|x_m) & P(y_2|x_m) & \cdots & P(y_n|x_m) \end{bmatrix}$$

其中每一行表示在某一个输入符号条件下，各输出符号出现的概率，因此每一行概率之和应满足：

$$\sum_{j=1}^n P(y_j|x_i) = 1$$

对于给定的输入分布 $P(x_i)$ ，输出分布为：

$$q(y_j) = \sum_{i=1}^m P(x_i)P(y_j|x_i)$$

输入与输出之间的互信息为：

$$I(X;Y) = \sum_{i=1}^m \sum_{j=1}^n P(x_i)P(y_j|x_i) \log_2 \frac{P(y_j|x_i)}{q(y_j)}$$

信道容量定义为所有可能输入分布下互信息的最大值：

$$C = \max_{P(x)} I(X;Y)$$

也就是说，信道容量不是某一个固定输入分布下的互信息，而是在选择最佳信源分布时，信道能够传输的最大平均信息量。

本实验采用迭代算法求解一般离散信道容量。程序首先将输入分布初始化为均匀分布，然后

根据当前输入分布计算输出分布 $q(y)$ ，再计算每个输入符号对应的相对信息量：

$$D_i = \sum_{j=1}^n P(y_j | x_i) \log_2 \frac{P(y_j | x_i)}{q(y_j)}$$

之后根据信息量更新输入分布：

$$p_i^{new} = \frac{p_i 2^{D_i}}{\sum_{k=1}^m p_k 2^{D_k}}$$

通过不断迭代，使输入分布逐渐趋近于最佳信源分布。当相邻两次迭代的信道容量变化很小，且输入分布变化也很小时，认为算法收敛。

程序并不是直接求出信道容量，而是从一个初始输入分布开始，不断调整各输入符号的概率。如果某个输入符号对区分输出更有贡献，它在后续迭代中的概率可能会增加；如果某个输入符号提供的信息较少，它的概率可能会减小。最终得到的输入分布就是使互信息最大的最佳信源分布。

三、实验内容

本实验主要完成以下内容：

1. 从键盘输入信源符号个数 m 和信宿符号个数 n 。
2. 输入信道转移概率矩阵 $P(y|x)$ 。
3. 检查输入矩阵是否合法，包括概率是否在 0 到 1 之间，以及每一行概率之和是否为 1。
4. 将初始信源分布设为均匀分布。
5. 根据迭代算法不断更新输入分布。
6. 在每次迭代中计算输出分布、互信息和信道容量近似值。
7. 判断算法是否收敛。
8. 输出最佳信源分布、对应输出分布、信道容量和迭代次数。
9. 绘制信道容量迭代收敛曲线，观察算法收敛过程。

四、程序代码

```
clc;
clear;
close all;

% 实验四：一般离散信道容量迭代算法

% 输入信道参数

m = input('请输入信源符号个数 m: ');
n = input('请输入信宿符号个数 n: ');

if m <= 0 || n <= 0 || floor(m) ~= m || floor(n) ~= n
    error('信源符号个数和信宿符号个数必须为正整数。');
end

P = zeros(m, n);

fprintf('\n 请输入信道转移概率矩阵 P(y|x): \n');
```

```
fprintf('每一行表示一个输入符号到各输出符号的转移概率\n');
fprintf('例如: [0.7 0.3]\n\n');

for i = 1:m
    row = input(sprintf('请输入第 %d 行的 %d 个转移概率: ', i, n));

    if length(row) ~= n
        error('第 %d 行输入的概率个数不等于 %d。', i, n);
    end

    P(i, :) = row;
end

% 检查转移概率矩阵是否合法

if any(P(:) < 0) || any(P(:) > 1)
    error('转移概率必须在 0 到 1 之间。');
end

for i = 1:m
    row_sum = sum(P(i, :));
    if abs(row_sum - 1) > 1e-6
        error('第 %d 行概率之和不等于 1, 请检查输入。', i);
    end
end

% 初始化参数

% 初始信源分布, 采用均匀分布
p = ones(1, m) / m;

% 最大迭代次数
max_iter = 10000;

% 收敛精度
epsilon = 1e-10;

% 记录容量变化
C_old = 0;
C_record = zeros(1, max_iter);

% 迭代计算信道容量

for iter = 1:max_iter

    % 计算输出分布 q(y)
    q = p * P;

    % 计算 D_i
    D = zeros(1, m);

    for i = 1:m
        for j = 1:n
            if P(i, j) > 0 && q(j) > 0
                D(i) = D(i) + P(i, j) * log2(P(i, j) / q(j));
            end
        end
    end
end
```

```

end

% 更新信源分布
temp = p .* (2.^D);
p_new = temp / sum(temp);

% 计算新的输出分布
q_new = p_new * P;

% 计算当前互信息
C_new = 0;

for i = 1:m
    for j = 1:n
        if P(i,j) > 0 && q_new(j) > 0
            C_new = C_new + p_new(i) * P(i,j) * ...
                log2(P(i,j) / q_new(j));
        end
    end
end

% 记录每次迭代的容量
C_record(iter) = C_new;

% 判断是否收敛
if abs(C_new - C_old) < epsilon && max(abs(p_new - p)) < epsilon
    break;
end

% 更新变量
p = p_new;
C_old = C_new;
end

if iter == max_iter
    warning('达到最大迭代次数，结果可能尚未完全收敛。');
end

% 输出结果

fprintf('\n 迭代计算结果\n\n');

fprintf('信道转移概率矩阵 P(y|x): \n');
disp(P);

fprintf('迭代次数: %d\n\n', iter);

fprintf('最佳信源分布: \n');
for i = 1:m
    fprintf('P(x%d) = %.10f\n', i, p_new(i));
end

fprintf('\n 对应输出分布: \n');
for j = 1:n
    fprintf('P(y%d) = %.10f\n', j, q_new(j));
end

```

```
fprintf('\n 信道容量 C = %.10f bit/symbol\n', C_new);

% 绘制信道容量收敛曲线

figure;
plot(1:iter, C_record(1:iter), 'LineWidth', 2);
grid on;
xlabel('迭代次数');
ylabel('信道容量近似值 C');
title('信道容量迭代收敛曲线');
```

五、实验结果及分析

1. 程序运行结果分析

本次实验输入的信道转移概率矩阵为：

$$P(y|x) = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.1 & 0.3 & 0.6 \end{bmatrix}$$

其中，信源符号个数为： $m = 2$

信宿符号个数为： $n = 3$

程序运行后，得到如下结果：

```
命令行窗口
请输入信源符号个数 m: 2
请输入信宿符号个数 n: 3

请输入信道转移概率矩阵 P(y|x):
每一行表示一个输入符号到各输出符号的转移概率
例如: [0.7 0.3]

请输入第 1 行的 3 个转移概率: [0.5 0.3 0.2]
请输入第 2 行的 3 个转移概率: [0.1 0.3 0.6]

迭代计算结果

信道转移概率矩阵 P(y|x):
    0.5000    0.3000    0.2000
    0.1000    0.3000    0.6000

迭代次数: 67

最佳信源分布:
P(x1) = 0.4822327015
P(x2) = 0.5177672985

对应输出分布:
P(y1) = 0.2928930806
P(y2) = 0.3000000000
P(y3) = 0.4071069194

信道容量 c = 0.1806954437 bit/symbol
```

图 4-1 程序运行结果

图 4-1 为 MATLAB 命令窗口中的程序运行结果。从结果可以看出，程序正确读取了信源符号个数、信宿符号个数以及信道转移概率矩阵，并经过 67 次迭代后得到最佳信源分布和信道容量。

最佳信源分布为：

$$P(x_1) = 0.4822327015$$

$$P(x_2) = 0.5177672985$$

可以看出，最佳信源分布不是完全均匀分布。虽然初始输入分布设置为：

$$P(x_1) = P(x_2) = 0.5$$

但是经过迭代后，程序将 x_2 的概率略微提高，将 x_1 的概率略微降低。这说明对于该信道来说，使互信息达到最大的输入分布并不是简单的均匀分布，而是与信道转移概率矩阵有关。

对应输出分布为：

$$P(y_1) = 0.2928930806$$

$$P(y_2) = 0.3000000000$$

$$P(y_3) = 0.4071069194$$

其中 $P(y_3)$ 最大，说明在最佳输入分布下，输出符号 y_3 出现的概率最高。

从信道转移矩阵可以看出：

$$P(y_2 | x_1) = 0.3, P(y_2 | x_2) = 0.3$$

无论输入为 x_1 还是 x_2 ，输出为 y_2 的概率都相同，因此 y_2 对区分输入符号的作用较弱。而对于 y_1 和 y_3 ：

$$P(y_1 | x_1) = 0.5, P(y_1 | x_2) = 0.1$$

$$P(y_3 | x_1) = 0.2, P(y_3 | x_2) = 0.6$$

二者在不同输入条件下差异较大，因此 y_1 和 y_3 更有助于判断输入符号。也正是因为该信道的输出分布仍然存在重叠，所以信道容量不高。

本次实验计算得到信道容量为：

$$C = 0.1806954437 \text{ bit/symbol}$$

该值大于 0，说明该信道具有传输信息的能力；但该值明显小于理想二输入无噪信道的容量：

$$\log_2 2 = 1 \text{ bit/symbol}$$

说明该信道存在较明显的不确定性和噪声，接收端不能完全根据输出符号判断输入符号。

2. 信道容量迭代收敛曲线分析

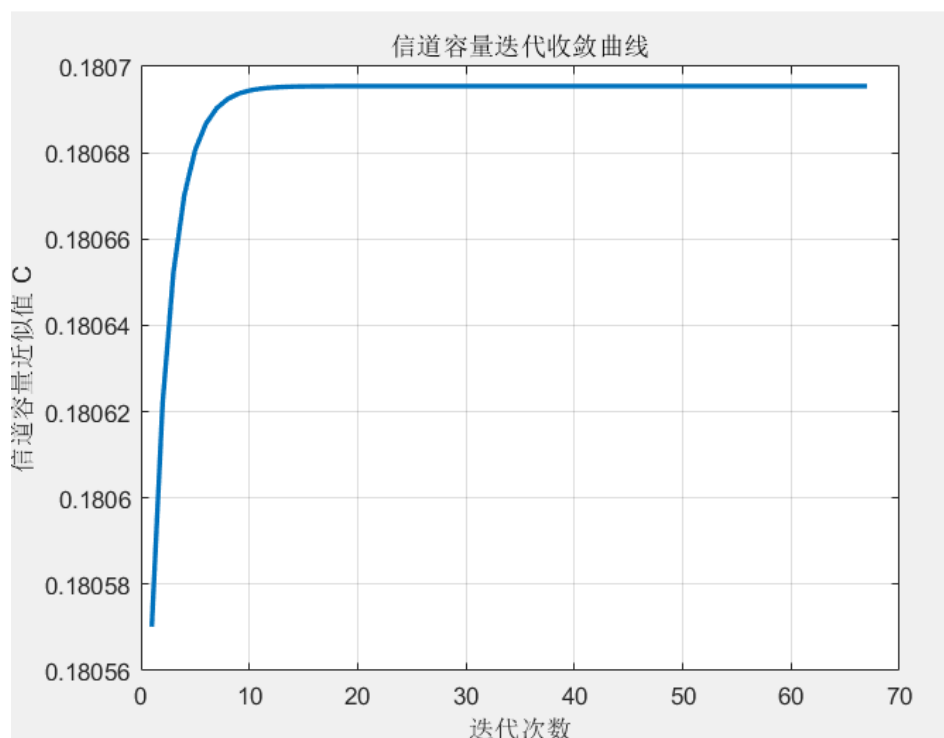


图 4-2 信道容量迭代收敛曲线

图 4-2 为信道容量随迭代次数变化的收敛曲线。由图可以看出，随着迭代次数增加，信道容量近似值 C 逐渐增大并最终趋于稳定。

在迭代初期，曲线上升较快，说明初始均匀输入分布经过调整后，互信息明显增大。随着迭代继续进行，曲线变化逐渐变缓，说明输入分布逐渐接近最佳信源分布。当迭代到一定次数后，曲线几乎保持水平，表示容量近似值已经基本不再变化，算法达到收敛状态。

本次实验中程序共迭代 67 次，最终得到：

$$C = 0.1806954437 \text{ bit/symbol}$$

从图中可以看出，迭代过程没有出现振荡或发散现象，说明该迭代算法在本实验信道下具有较好的收敛性。信道容量迭代算法的本质是不断寻找最优输入分布，使互信息逐步接近最大值。曲线最终趋于平稳，说明程序已经找到了较稳定的最佳输入分布和信道容量。

3. 算法与结果综合分析

本实验中，最佳信源分布为：

$$P(x_1) = 0.4822327015, P(x_2) = 0.5177672985$$

它与均匀分布非常接近，但并不完全相同。这说明对于某些信道，均匀输入分布可能接近最优，但不一定就是最优。信道容量的求解必须根据转移概率矩阵，通过迭代算法获得。

本实验信道容量为：

$$C = 0.1806954437 \text{ bit/symbol}$$

这个数值较小，主要原因是信道存在较强的随机性。两个输入符号对应的输出概率分布并不是完全分离的，而是存在重叠。例如输出 y_2 在两个输入条件下的概率完全相同，因此它不能提供区分输入符号的信息。

如果两个输入符号对应的输出分布差异越大，接收端越容易根据输出判断输入，信道容量就越大；如果不同输入符号对应的输出分布越接近，信道容量就越小。当所有输入符号对

应的输出分布完全相同时，输出与输入无关，此时信道容量为 0。

因此，本次实验结果很好地说明了信道容量的物理意义：信道容量反映的不是信道中有多少个输入或输出符号，而是信道在最佳输入分布下能够可靠传递信息的最大能力。

六、实验结论及体会

通过本次实验，我利用 MATLAB 实现了一般离散信道容量的迭代算法。程序能够根据输入的任意信道转移概率矩阵，计算最佳信源分布、对应输出分布和信道容量，并绘制信道容量的迭代收敛曲线。

本次实验输入的信道转移概率矩阵为：

$$P(y|x) = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.1 & 0.3 & 0.6 \end{bmatrix}$$

经过 67 次迭代后，得到最佳信源分布为：

$$P(x_1) = 0.4822327015$$

$$P(x_2) = 0.5177672985$$

对应的信道容量为：

$$C = 0.1806954437 \text{ bit/symbol}$$

实验结果表明，最佳信源分布不一定是均匀分布，而是由信道转移概率矩阵决定的。通过迭代算法，可以逐步调整输入符号的概率，使互信息不断接近最大值，最终得到信道容量。

通过观察信道容量收敛曲线，我更加直观地理解了迭代算法的过程。算法不是一步得到最终结果，而是从初始输入分布出发，经过多次更新逐渐收敛。曲线最终趋于平稳，说明互信息已经接近最大值，程序计算结果可靠。

本实验也让我进一步理解了信道容量的物理意义。信道容量表示信道在最佳输入分布下能够传输信息的最大平均信息量。它不仅与输入、输出符号个数有关，更主要取决于信道转移概率矩阵。如果不同输入符号对应的输出分布差异较大，信道容量就较大；如果输出分布相互重叠严重，信道容量就较小。

信道转移概率矩阵中每一行都必须表示一个完整的条件概率分布，因此每一行概率之和必须为 1，且每个转移概率都必须在 0 到 1 之间。如果不进行检查，程序可能会得到错误的计算结果。

总体来说，本次实验加深了我对一般离散信道容量、最佳信源分布和迭代算法的理解。同时，通过 MATLAB 编程实现输入检查、迭代计算、结果输出和收敛曲线绘制，也提高了我用程序分析信息论问题的能力。